［問題５］

n以下の素数を、次のようにしてすべて求めたいと思います（但し、nは自然数）。

まず、1を除いたn以下の自然数をすべて作ります。そして、先頭の数の倍数を、その数自身を除いてすべて消去します。次に、消されずに残った数の先頭の数の倍数をすべて消去します。以下、同様にして消去する数がなくなるまでこれを繰り返せば、残った数が素数になります。

例えば、40以下の素数を求める過程を示すと、次のようになります。

２の倍数の消去

2 3 ~~4~~ 5 ~~6~~ 7 ~~8~~ 9 ~~10~~ 11 ~~12~~ 13 ~~14~~ 15 ~~16~~ 17 ~~18~~ 19 ~~20~~
21 ~~22~~ 23 ~~24~~ 25 ~~26~~ 27 ~~28~~ 29 ~~30~~ 31 ~~32~~ 33 ~~34~~ 35 ~~36~~ 37 ~~38~~ 39 ~~40~~

３の倍数の消去

2 3 ~~4~~ 5 ~~6~~ 7 ~~8~~ ~~9~~ ~~10~~ 11 ~~12~~ 13 ~~14~~ ~~15~~ ~~16~~ 17 ~~18~~ 19 ~~20~~
~~21~~ ~~22~~ 23 ~~24~~ 25 ~~26~~ ~~27~~ ~~28~~ 29 ~~30~~ 31 ~~32~~ ~~33~~ ~~34~~ 35 ~~36~~ 37 ~~38~~ ~~39~~ ~~40~~

５の倍数の消去

2 3 ~~4~~ 5 ~~6~~ 7 ~~8~~ ~~9~~ ~~10~~ 11 ~~12~~ 13 ~~14~~ ~~15~~ ~~16~~ 17 ~~18~~ 19 ~~20~~
~~21~~ ~~22~~ 23 ~~24~~ ~~25~~ ~~26~~ ~~27~~ ~~28~~ 29 ~~30~~ 31 ~~32~~ ~~33~~ ~~34~~ ~~35~~ ~~36~~ 37 ~~38~~ ~~39~~ ~~40~~

これ以上消去できないので、残った

2 3 5 7 11 13 17 19 23 29 31 37

が素数

これは**「エラトステネスのふるい」**と呼ばれる、素数を非常に効率よく求めるアルゴリズムです。以下に、このアルゴリズムを作成する段階的詳細化の過程を、疑似コードで途中まで示してみます。

必要な処理は、入力された自然数nに対してn以下の自然数を作り、次々に倍数を消去し、最後に残った自然数を表示することけですから、アルゴリズムの大枠は次のような構成になるでしょう。

```
//  自然数ｎ以下の素数をすべて求める

main()
{
        1. 自然数ｎの入力;
        2. １を除くｎ以下の自然数を用意する;
        3. 倍数を次々に消去していく;
        4. 残った自然数（素数）を表示する;
}
```

まず、[3. 倍数を次々に消去していく]部分から詳細化することにします。この部分は、ある自然数iの倍数をまだ消去していなければ、iの倍数をすべて消去する、という処理の繰り返しになります(但し、i自身は消去しない)。この処理は、倍数を消去していない自然数iが残っているあいだ繰り返せばよいわけですから、繰り返し構造と選択構造を使って次のようになります。

```
// 3. 倍数を次々に消去していく

while (倍数を消去していない自然数 i が残っている) {
    if (自然数 i の倍数をまだ消去していない) {
        3.1 自然数 i の倍数をすべて消去する;
    }
    3.2 次の自然数 i を選ぶ;
}
```

ここで、自然数iの倍数を消去するという操作は、適当な配列を用意し、そのi番目に0(零)を入れることにします。すると、[3.1 自然数iの倍数をすべて消去する]部分は、繰り返し構造を用いて次のように詳細化できます。ここでは、natural[] という配列を使うことにします。

```
// 3.1 自然数 i の倍数をすべて消去する

j = i の倍数で最初に消去する自然数;
while (j <= n) {
    natural[j] = 0;
    j = 次に消去する i の倍数;
}
```

さて、[2. 1を除くn以下の自然数を用意する]部分を考えてみます。倍数の消去は配列 natural[] に0を入れることで実現したので、この部分は、配列 natural[] に0以外の値を入れておき、配列の添え字がその自然数を表すことにすればよいことになります。0以外の値は何でもよいわけですが、ここでは1を入れることにすると、繰り返し構造を用いて以下のように詳細化できます。

```
// 2. 1を除く n 以下の自然数を用意する

for (i = 2; i <= n; i++) {
    natural[i] = 1;
}
```

ここまでの詳細化により、配列 natural[] の内容は、その添え字(配列の番号)が素数ならば1に、素数でなければ0になっているはずです。ですから、[**4. 残った自然数(素数)を表示する**]という部分を詳細化すると、繰り返し構造と選択構造を使って、次のようになります。

```
// 4. 残った自然数（素数）を表示する

for (i = 2; i <= n; i++) {
    if (natural[i] == 1) {
        添え字 i を表示；
    }
}
```

(1) ここまでの詳細化の各過程を対応する各部分に代入してみなさい。

(2) (1)の結果にさらに詳細化を進め完全なプログラムにし、コンパイル，実行しなさい。
ただし、自然数nの最大値は 1000000(=10^6) とします。

(prog05-02.c)

(3) 完成したプログラムを高速化する方法を考え、プログラムを改良しなさい。

(prog05-03.c)

[ヒント]

自然数iの倍数を消去する際に、iはn以下の自然数全てについて考える必要はありません(例えば100以下の自然数を考えるとき、15や16の倍数は既に5や4の倍数として消去されているので、15や16の倍数は消去の対象として考える必要なないわけです)。どの自然数までを、倍数を消去する対象とすればよいかを考えなさい。

(参考1) 実行時間の計測

高速化できたかどうかは、unixの **timeコマンド** を用いて、実行時間を計測することで確認できます。

ただし、プログラムの実行において、入出力はCPUが行う計算よりも非常に長い時間がかかります(つまり入出力は遅い)。そのため、全処理の中に占める入出力の割合が多いような場合は、CPUが実際に行った処理時間が入出力時間の中に埋もれてしまい、その正確な時間を計ることができません。特に、この「エラトステネスのふるい」のように非常に高速なアルゴリズムではそれが顕著になります。

その場合、printf()関数などで画面表示をする部分は、 /* printf(…) */ のようにコメントアウトして表示を抑制し、その後再コンパイルしてtimeコマンドで時間を測定するようにします。

timeコマンドは次のように使います。

```
$ time a.out ↵
n: 1000000 ↵

real    0m2.186s  ←  入出力も含めて実行にかかった時間
user    0m0.027s  ←  CPUでの処理にかかった時間(今回はこの時間を読む)
sys     0m0.004s  ←  OSのオーバヘッド
```

（参考2） ソースの修正なしで実行プログラム(バイナリ)を変更する方法

（参考1)では、画面表示する部分をコメントアウトして再コンパイルを行うようにしていましたが、次のようにすると、ソースプログラムに手を加えることなく、コンパイルオプションだけで表示を制御することができます。

① ソースプログラム内の表示部分を #ifndef NO_DISP ～ #endif で囲むように修正します(この修正は一度だけ行えばよい)。

```
// 4. 残った自然数（素数）を表示する

#ifndef NO_DISP                ←  追加
for (i = 2; i <= n; i++) {
    if (natural[i] == 1) {
        添え字 i を表示;
    }
}
#endif                         ←  追加
```

② 表示するかどうかで、次のようにコンパイル方法を使い分けます。

(a)画面表示する場合　…　通常通りコンパイル

```
$ cc prog05-0x.c ↵
$ a.out ↵  ←  このバイナリの実行結果は画面表示される
```

(b)画面表示させない場合　…　"-D NO_DISP"オプション付きでコンパイル

```
$ cc -D NO_DISP prog05-0x.c ↵
$ a.out ↵  ←  このバイナリの実行結果は画面表示されない
```

［問題６］

［問題5］では配列を用いたため、大きな配列が用意できない場合には大きな素数を求めることができません。そこで、自然数n以下の素数を、配列を使わない別の方法で求めてみたいと思います。以下の各問に答えながらプログラムを完成させなさい。

（1）次は、このアルゴリズムを作成する段階的詳細化の過程を、疑似コードで示したものです。空欄を埋めなさい。

素数は2を除いてすべて奇数ですから、奇数を小さい方から1つずつ素数かどうか順番にnまで調べていくことにします。すると、アルゴリズム全体は次のような逐次構造になります。

```
//  自然数ｎ以下の素数を求める
main()
{
    1. 自然数ｎを入力する;
    2. 素数２をそのまま表示する;
    3. ｎ以下の奇数を素数かどうか順に調べ、素数なら表示する;
}
```

これから、詳細化をさらに進めていきます。
まず、[3. ｎ以下の奇数を素数かどうか順に調べ、素数なら表示する]という部分は、繰り返し構造と順次構造を用いて次のようにすればよいでしょう。

```
//  3. ｎ以下の奇数を素数かどうか順に調べ、素数なら表示する
for (i = [  (1)  ] ; i <= n; [  (2)  ] ) {
    3.1 奇数ｉが素数かどうか調べる;
    3.2 奇数ｉが素数なら ｉ を表示する;
}
```

このとき奇数は、

1, 3, 5, 7, ･･･

となっているので、1から始めて順に2ずつ増やしていけば作ることができますが、1は素数ではないので ｉ の初期値をいくつにするかに注意して下さい。

次に、[3.1 奇数 ｉ が素数かどうか調べる]という部分を考えます。ある数が素数かどうかは、その数を2以上の自然数で順に割っていき、その数自身以外の数で割り切れるかどうか調べることによって知ることができます（つまり、自分自身以外で割り切れなければその数は素数）。

しかし、ここで素数かどうかを調べる対象となっている数は奇数ですから、2の倍数（すなわち偶数）では絶対に割り切れません。ということは、その数より小さい3以上の奇数で

割り切れるかどうかを順に調べればよいわけです(もちろん、1で割り切れるかどうかを調べる必要は当然ありません)。したがって、この部分のアルゴリズムは、素数かどうか調べる対象となっている数(奇数)を、「自分自身より小さい3以上の奇数で割り切れるかどうか順に調べる」ということを繰り返す、次のような構造にすればよいことになります。

```
// 3.1 奇数 i が素数かどうか調べる
j = [  (3)  ] ;
while ((3.1.1 jがiより小さい) && (3.1.2 iがjで割り切れない)) {
      j = [  (4)  ] ;
}
```

ここで、[3.1.2 iがjで割り切れない]という部分は、余りを求める演算子 **%** を使用した次のような条件式になります(確認: A % B は A を B で割った余りを計算します。例えば 13 % 5 は 3 になります)。

[(5)]

最後に、[3.2 奇数 i が素数なら i を表示する]という部分は、[3.1]のwhileループを終了したとき、i と j が [(6)] という関係を満たしていれば i は素数であるわけですから、次のような構造になります。

```
// 3.2 奇数 i が素数なら i を表示する
if ( [  (6)  ] ) {
    3.3 素数を表示する;
}
```

これで、入出力の部分を除き、この問題の詳細化の大部分は完成しました。

(2) (1)の詳細化の各過程を代入して得られる結果を疑似コードで表現しなさい。

(3) (2)の結果を完全なC言語のプログラムに直し、コンパイル，実行しなさい。
ただし、自然数nの最大値は 1000000000(10^9) とします。 (prog06-03.c)

(4) [問題5]と同様に(3)のプログラムを改良し、高速化しなさい。
なお、このアルゴリズムは実行全体に占めるCPUの処理時間大きいため、実行時間を測定するのに表示の抑制をする必要はありません。 (prog06-04.c)

[ヒント]

ある数が素数かどうかを調べるときに、(1)ではその数より小さいすべての奇数について割り切れるかどうかを調べました。しかし、その数よりずっと小さい奇数までについて調べるだけで済みます。いくつまで調べればよいか考えなさい。

(実はその数の平方根までの奇数を調べればよいのですが、理由は[問題5](3)のヒントを参考に各自考えなさい)。

［問題7］

次の(1)(2)のプログラムを段階的詳細化の手法を用いて作りなさい。また、設問がある場合はそれにも答えなさい。

（１）分数の表示　(prog07-01.c)

2つの正の整数M，Nが与えられたとき、Mを分母、Nを分子とする分数 $\frac{N}{M}$ を次の形式で表示するプログラム

○ $\frac{N}{M}$ が整数aになるときは、

N／M＝a

と表示。

○ $\frac{N}{M}$ が1より小さいときは、既約分数a／bに直して

N／M＝a／b

と表示。

○ $\frac{N}{M}$ が1より大きい非整数のときは、既約な帯分数に直して

N／M＝k(a／b)

と表示。

(注:既約分数とは、それ以上約分のできない分数のことです)

［ヒント］

分数 $\frac{N}{M}$ を既約分数にするために、MとNの最大公約数を見つける必要があります。最大公約数を見つけるアルゴリズムとしてよく知られたものに、ユークリッドの互助法というものがあります。

（参考）ユークリッドの互除法

整数MとNの最大公約数は、以下のような手順で求めることができます。
但し、M＞Nであるものとします。

① MをNで割った余りRを求める。
② R＝0のとき、Nの値が最大公約数。
③ R≠0のとき、NをMへ、RをNに代入し(1)へ戻る。

（２）文字列の探索　(prog07-02.c)

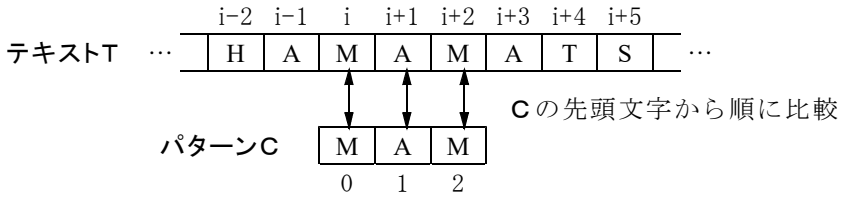
長さ m のテキスト T と長さ n のパターン C が与えられたとき、テキスト T の中からパターン C を見つけ出し、最初に見つかった位置を変数 P に格納するプログラム。

ただし、T と C はいずれも文字配列に格納された文字列で、1 文字ずつそれぞれ $T_0, T_1, \cdots, T_{m-1}$、および $C_0, C_1, \cdots, C_{n-1}$ に格納されており、$0 \leqq n \leqq m \leqq 10^6$ であるします。また、変数 P に格納する値は、

パターンCが見つかったとき　…　C_0 と一致したTの文字番号
パターンC見つからなかったとき　…　−1

とします。例えば、m = 16 で、$T_0 \sim T_{15}$ = "HAMAMATSUKOUGYOU" のとき、長さ n = 3 のパターン C が C_0, C_1, C_2 = "MAMA" ならば P = 2 に、"ABC"なら P = -1 になります。

なお、一致するかどうかは、「テキスト T の i 番目の文字 T_i とパターン C の先頭文字 C_0 とを比較し、一致すればそれぞれ比較する文字の位置を次に進める」という操作で行うことにします（下図参照）。



（設問Ａ）このプログラムに用いたアルゴリズムの最大計算量のオーダーを、m と n で表しなさい。また、m と n がともに 2 倍になると、計算量は何倍になりますか。

（設問Ｂ）このプログラムを高速化するにはどうすればいいと思いますか。どんな方法があるか考え、そのアイデアを説明して下さい。
（ヒント：常に前から比較しなければならないでしょうか？）

レポート

次のものをE-mailで提出しなさい。

・[問題5]（2）（3）のプログラム（添付ファイル）と実行結果（本文）
・[問題6]（3）（4）のプログラム（添付ファイル）と実行結果（本文）
・[問題7]（1）（2）のプログラム（添付ファイル）と実行結果（本文）
　注意：段階的詳細化をどのように進めたか、その過程がわかるようにプログラム内にしっかりコメントを書いておくこと。
・[問題7]（2）の（設問A，B）の解答（本文）
・演習の感想（本文）

（提　出　先）k-kenta@hamako-ths.ed.jp　件名：プロ技演習５～７
（提出期限）　　／　　（　　）